Estruturas de Dados I

Apresentação da Disciplina e Introdução à Linguagem C

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

6 de agosto de 2026

Um pouco sobre mim...

2012–2016 **Graduação em Ciência da Computação (UEL)**
TCC: *Simulação numérica do problema de dispersão de poluentes na atmosfera*

2016–2018 **Mestrado em Ciência da Computação (UEL)**
Dissertação: *Geração e análise de qualidade de malhas computacionais em coordenadas generalizadas*

2018–2022 **Doutorado em Modelagem Computacional (LNCC)**
Tese: *Hybrid Modeling Framework for Tumor Growth with Chemotherapy*

Um pouco sobre mim...

2022–2025 **Pós-Doutorado (LNCC)**
Projeto: *Abordagem de Modelagem Híbrida para o Crescimento Tumoral com Tratamento*

2024–2025 **Orientação de Estágio (UniFil)**

2025–Atual **Professor Temporário (UEL)**

2026–Atual **Game Designer (Firepot Gaming)**

Objetivos da Aula

- Apresentar a disciplina, sua ementa e critérios de avaliação.
- Diferenciar linguagens interpretadas de linguagens compiladas.
- Identificar a estrutura básica de um programa na linguagem C.
- Compilar e executar um programa em C.

Apresentação da Disciplina

Ementa da Disciplina

- Tipos Abstratos de Dados (TADs).
- Formas de armazenamento: estática e dinâmica (utilização de ponteiros).
- Listas lineares: pilhas, filas e listas.
- Recursividade: propriedades da recursão e noções de complexidade de algoritmos.
- Árvores: conceitos, árvores binárias e árvores n-árias.
- Busca e ordenação: busca binária, árvore de busca binária, *Bubble Sort*, *Insertion Sort*, *Merge Sort*, *Quick Sort* e *Heap Sort*.

Critérios de Avaliação

3° Bimestre

- Prova teórica: **70%**;
- Exercícios: **30%**.

4° Bimestre

- Prova teórica: **50%**;
- Trabalho prático (com ou sem seminário): **30%**;
- Exercícios: **20%**.

Bibliografia

Básica

- TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J. *Estruturas de Dados Usando C*. São Paulo: Pearson Makron Books.
- CELES, W.; CERQUEIRA, R.; RANGEL, J. L. *Introdução a Estruturas de Dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier.
- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Prática*. Rio de Janeiro: LTC.

Bibliografia

Complementar

- ZIVIANI, N. *Projeto de Algoritmos com Implementações em Pascal e C*. São Paulo: Cengage Learning.
- KERNIGHAN, B. W.; RITCHIE, D. M. *C: A Linguagem de Programação*. Rio de Janeiro: Campus.
- BACKES, A. *Estrutura de Dados Descomplicada em Linguagem C*. Rio de Janeiro: Elsevier.
- SEDGEWICK, R. *Algorithms in C*. Boston: Addison-Wesley.

Linguagens Interpretadas × Compiladas

Execução de um Programa

- O processador só é capaz de executar instruções em **linguagem de máquina**: sequências de bits que representam operações elementares.
- O código que escrevemos em uma **linguagem de programação** é texto voltado à leitura humana.
- Entre um e outro existe uma etapa de **tradução**.
- Essa tradução define linguagens **interpretadas** e linguagens **compiladas**.

Linguagens Interpretadas

- Um programa auxiliar, o **interpretador**, lê o código-fonte e executa cada instrução no momento em que a encontra.
- Não existe um arquivo executável.
- Para executar o programa em outra máquina, é necessário que o interpretador esteja instalado nela.
- Erros só se manifestam quando a linha problemática é alcançada.

Linguagens Compiladas

- Um programa auxiliar, o **compilador**, traduz todo o código-fonte antes de qualquer execução.
- O resultado é um arquivo executável em linguagem de máquina.
- O executável roda diretamente no sistema operacional, sem precisar do compilador.
- Erros de sintaxe e de tipo são detectados na compilação.

Linguagens Interpretadas × Compiladas

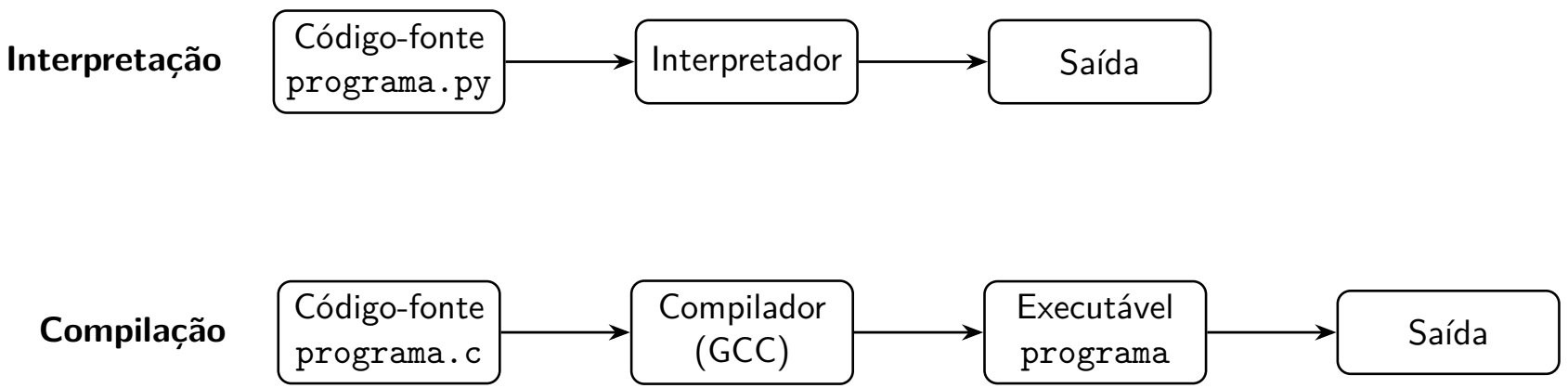
Figura 1: Linguagens interpretadas × compiladas



Fonte: https://www.techtrainghq.com/interpreted-vs-compiled-programming-languages-difference/

Fluxos de Execução

Figura 2: Tradução do código-fonte em uma linguagem interpretada e em uma linguagem compilada.



Detecção de Erros

- Um trecho nunca executado passa despercebido em uma linguagem interpretada:

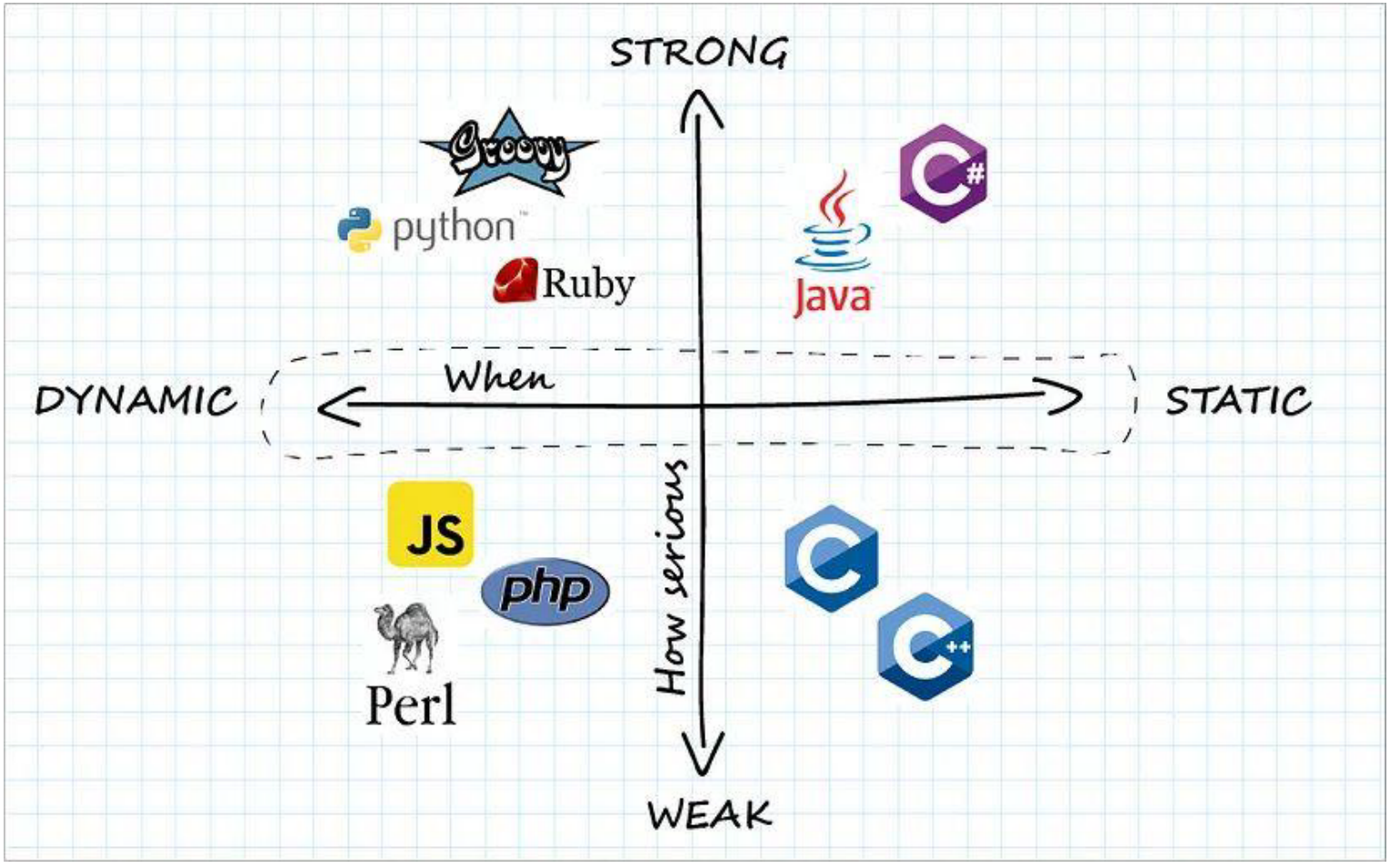
```python
# Python
if False:
    print(soma)
print("Fim")
```

- Em uma linguagem compilada, o mesmo trecho impede a geração do executável:

```c
// C
if (0) {
    printf("%f\n", soma);
}
printf("Fim\n");
```

Comparação de Linguagens por Tipagem

Figura 3: Comparação de linguagens por tipagem



Tipagem Dinâmica × Estática

- Na **tipagem dinâmica**, o tipo pertence ao valor e é verificado durante a execução. Uma mesma variável pode guardar valores de tipos diferentes:

```python
# Python
x = 10
x = "dez"
```

- Na **tipagem estática**, o tipo é declarado, fixo e verificado na compilação:

```c
// C
int x = 10;
x = "dez"; // Erro
```

- Declarar o tipo informa ao compilador quantos *bytes* reservar e como interpretá-los.

Tipagem Forte × Fraca

- Na **tipagem forte**, operações entre tipos incompatíveis são rejeitadas:

```python
# Python
4 + "5" # TypeError: unsupported operand type(s)
4 + int("5") # 9
```

- Na **tipagem fraca**, a linguagem converte os valores por conta própria e permite reinterpretar os bits armazenados:

```c
// C
int x = 'A' + 1; // 66
int y = 3.9; // 3
```

- No exemplo anterior, nenhum dos dois casos interrompe a compilação.

Gerenciamento de Memória

- Em Python, a reserva e a liberação de memória são automáticas, conduzidas por um *garbage collector*.
- O programador não vê endereços de memória, e o mecanismo funciona como uma "caixa-preta".
- Em C, a memória é solicitada e devolvida explicitamente pelo programador.
- Isso implica mais responsabilidade e o risco de vazamentos de memória, mas oferece controle e previsibilidade sobre o custo das operações.

Python × C

Tabela 1: Principais diferenças entre Python e C.

Aspecto	**Python**	**C**
Tradução	Interpretada	Compilada
Tipagem	Dinâmica e Forte	Estática e Fraca
Declaração de variáveis	Dispensável	Obrigatória
Gerenciamento de memória	Automático	Manual
Delimitação de blocos	Indentação	Chaves { }
Fim de instrução	Quebra de linha	Ponto e vírgula
Detecção de erros	Durante a execução	Durante a compilação
Desempenho	Menor	Maior
Nível de abstração	Alto	Intermediário

Introdução à Linguagem C

Por que C em Estruturas de Dados?

- O objetivo da disciplina não é apenas usar estruturas de dados, mas construí-las.
- Em linguagens de alto nível, listas, pilhas e dicionários vêm prontos e escondem como a memória é organizada.
- Em C, cada nó precisa ser alocado, conectado e liberado explicitamente, o que torna a estrutura visível.
- O custo de cada operação também fica visível, o que sustenta a análise de complexidade.

Instalando o Compilador GCC*

- **Linux**: instale o pacote com as ferramentas de compilação:

```
$ sudo apt update
$ sudo apt install build-essential
```

- **Mac OS**: instale o compilador por meio do Homebrew:

```
$ brew install gcc
```

- **Windows**: instale o MSYS2 (`https://www.msys2.org`) e, por meio dele, o pacote `mingw-w64-x86_64-gcc`.
- Para verificar a instalação, execute:

```
$ gcc --version
```

*GNU Compiler Collection

Ciclo de Trabalho

- **Criar/Editar**: escrever o código em um editor de texto e salvá-lo com a extensão `.c`.
- **Compilar**: gerar o executável a partir do código-fonte.

```
$ gcc programa.c -o programa
```

- **Executar**: rodar o arquivo gerado.

```
$ ./programa
```

- No Windows, o executável recebe a extensão `.exe`.
- Sem a opção `-o`, o GCC nomeia o executável como `a.out`.
- Toda alteração no código-fonte exige uma nova compilação.

Estrutura Geral de um Programa em C

- Todo programa em C parte de um esqueleto mínimo, sempre presente:

```c
#include <stdio.h>   // Bibliotecas

int main() {         // Função principal
    // Código aqui...

    return 0;        // Indica que o programa terminou com
     sucesso
}
```

Programa em C: Exemplo

```c
#include <stdio.h>

int main() {
    printf("Hello world!\n");
    return 0;
}
```

- Salvando o arquivo como `hello.c`, compilando e executando:

```
$ gcc hello.c -o hello
$ ./hello
Hello world!
```

Programa em C: Exemplo

```c
#include <stdio.h>                  // 1

int main() {                        // 2
    printf("Hello world!");         // 3
    return 0;                       // 4
}
```

1. **Diretiva de pré-processador:** torna disponíveis as funções de entrada e saída.
2. **Cabeçalho da função `main`:** ponto de partida da execução.
3. Instrução que produz a **saída**.
4. **Valor devolvido** ao sistema operacional ao final da execução.

Diretivas de Pré-processador

- Antes da compilação, um programa auxiliar chamado **pré-processador** percorre o código e executa as linhas iniciadas por #.
- A **diretiva `#include`** substitui a linha pelo conteúdo do arquivo indicado, disponibilizando as funções ali declaradas.
- **Exemplo:** A biblioteca `stdio.h` contém, entre outras, as funções `printf()` e `scanf()`.

Bibliotecas

- **Bibliotecas** são coleções de subprogramas prontos, escritos para serem reaproveitados no desenvolvimento de software.
- Reúnem **funções** que não fazem parte do núcleo da linguagem, mas que resolvem tarefas frequentes.
- Existem em **todas** as linguagens de programação.
- Os programas em C recorrem às funções da **biblioteca padrão**, disponível em qualquer compilador da linguagem.
- Existem também bibliotecas **desenvolvidas por terceiros**.

Bibliotecas Padrão

Tabela 2: Bibliotecas padrão da linguagem C mais utilizadas

Biblioteca	**Utilidades**
`<ctype.h>`	Funções para classificar e converter caracteres
`<float.h>`	Constantes e limites dos números de ponto flutuante (números reais)
`<math.h>`	Funções matemáticas
`<stdbool.h>`	Tipo booleano
`<stdio.h>`	Funções de entrada e saída
`<stdlib.h>`	Alocação dinâmica de memória, conversões e utilidades gerais
`<string.h>`	Funções para trabalhar com letras, palavras ou frases (*strings*)
`<time.h>`	Funções para trabalhar com data e hora

Função `main()`

- Todo programa em C possui exatamente **uma função `main()`**.
- O `int` antes do nome indica que a função retorna um número inteiro ao sistema operacional.
- Por convenção, `return 0` sinaliza que o programa terminou sem erros. Valores diferentes de zero indicam falha.
- As **chaves** delimitam o corpo da função.
- A **indentação** não altera o significado do código, mas é indispensável para a legibilidade.

Comentários

- **Comentários** são ignorados pelo compilador e servem para documentar o código.
- Comentário de **uma linha** é iniciado por //:

```
printf("Hello world!"); // Exibe a mensagem no console
```

- Comentários de **múltiplas linhas** são delimitados por /* e */:

```
/*
   Programa de exemplo
   Autor: Gustavo
   Disciplina: Estruturas de Dados I
*/
```

Erro de Compilação × Erro de Execução

- **Erro de compilação**: o código viola as regras da linguagem. O executável não é gerado, e o compilador indica o arquivo, a linha e a natureza do problema.

```
hello.c:4:32: error: expected ';' before 'return'
```

- **Erro de execução**: o código é válido, o executável é gerado, mas algo falha durante a execução.

```
$ ./hello
Segmentation fault (core dumped)
```

Erros Comuns de Sintaxe

- Ponto e vírgula ausente ao final da instrução:

```
printf("Hello world!") // error: expected ';'
return 0;
```

- Chave não fechada:

```
int main() {
    printf("Hello world!");
    return 0;
// error: expected declaration or statement at end of
    input
```

Erros Comuns de Sintaxe

- Ausência de `#include <stdio.h>` ao utilizar `printf()`:

```c
int main() {
    printf("Hello world!\n");
    return 0;
}
// error: implicit declaration of function 'printf'
```

Estruturas de Dados I

Estruturas de Repetição e Funções em C

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

26 de agosto de 2026

Objetivos da Aula

- Implementar **estruturas de repetição** com `while`, `do-while` e `for`.
- Reconhecer os **erros** mais frequentes na implementação de laços.
- Controlar o fluxo de repetição com **`break` e `continue`**.
- Definir e chamar **funções**, declarando os seus protótipos.
- Identificar o **escopo** de variáveis.
- Compreender a **passagem de parâmetros** por valor.

Comando `while`

Comando while

- O **comando while** repete um bloco de comandos enquanto a condição for verdadeira.
- **Sintaxe:**

```
while (condicao) {
    // Bloco repetido enquanto a condição for verdadeira
}
```

- Os parênteses ao redor da condição são **obrigatórios**.
- Assim como nas estruturas condicionais, a condição é uma **expressão inteira**.
- Sem as chaves, o `while` repete apenas a instrução **imediatamente seguinte**.

Comando while: Exemplo

- **Problema:** ler um número inteiro positivo n e imprimir todos os inteiros de 1 até n.

```c
#include <stdio.h>

int main() {
    int n, i = 1;
    printf("Digite um numero inteiro positivo: ");
    scanf("%d", &n);

    while (i <= n) {
        printf("%d\n", i);
        i = i + 1;
    }

    return 0;
}
```

Comando while: Exemplo

- **Problema:** ler um número inteiro positivo n e calcular $n!$.

```c
#include <stdio.h>

int main() {
    int n, i = 2, fat = 1;
    printf("Digite um numero inteiro positivo: ");
    scanf("%d", &n);

    while (i <= n) {
        fat *= i;
        i++;
    }

    printf("Resultado: %d\n", fat);
    return 0;
}
```

Erros Comuns

- Um ponto e vírgula após a condição encerra o comando:

```
int i = 1;
while (i <= 5); // Comando vazio: laço infinito
{
    printf("%d\n", i);
    i = i + 1;
}
```

- Sem chaves, apenas a primeira instrução pertence ao laço:

```
int i = 1;
while (i <= 5)
    printf("%d\n", i);
    i = i + 1; // Fora do laço: laço infinito
```

Erros Comuns

- Atribuição no lugar da comparação:

```
int i = 1;
while (i = 5) { // Laço infinito
    printf("%d\n", i);
    i = i + 1;
}
```

- Nenhum desses casos gera erro de compilação: a falha aparece apenas na execução.

Comando `do-while`

Comando do-while

- O **comando `do-while`** executa o bloco e só então avalia a condição.
- **Sintaxe:**

```
do {
    // Bloco repetido enquanto a condição for verdadeira
} while (condicao);
```

- O ponto e vírgula após a condição é **obrigatório**.
- Como o teste ocorre no final, o bloco é executado pelo menos **uma vez**, mesmo que a condição seja falsa desde o início.

Comando while × do-while

```c
int i = 10;

while (i < 5) {
    printf("%d\n", i); // Não há impressão
    i++;
}
```

```c
int i = 10;

do {
    printf("%d\n", i); // Imprime 10
    i++;
} while (i < 5);
```

Comando do-while: Exemplo

- **Problema:** ler uma nota entre 0 e 10, insistindo enquanto o valor informado for inválido.

```c
#include <stdio.h>

int main() {
    double nota;

    do {
        printf("Digite a nota (0 a 10): ");
        scanf("%lf", &nota);
    } while (nota < 0 || nota > 10);

    printf("Nota registrada: %.1lf\n", nota);
    return 0;
}
```

Comando do-while: Exemplo

- **Problema:** exibir um menu com as opções de cadastrar, consultar e sair, repetindo-o até que o usuário escolha encerrar o programa.

```c
#include <stdio.h>

int main() {
    int opcao;

    do {
        printf("\n1 - Cadastrar\n");
        printf("2 - Consultar\n");
        printf("0 - Sair\n");
        printf("Opcao: ");
        scanf("%d", &opcao);
        ...
```

Comando do-while: Exemplo

```
        ...
        switch (opcao) {
            case 1:
                printf("Cadastrando...\n");
                break;
            case 2:
                printf("Consultando...\n");
                break;
            case 0:
                printf("Encerrando...\n");
                break;
            default:
                printf("Opcao invalida\n");
        }
    } while (opcao != 0);
    return 0;
}
```

Comando `for`

Comando `for`

- Em C, o **comando `for`** não percorre os elementos de uma sequência: ele reúne, em um único cabeçalho, as três partes que controlam um laço.
- **Sintaxe:**

```
for (inicializacao; condicao; atualizacao) {
    // Bloco repetido enquanto a condição for verdadeira
}
```

- **Inicialização:** executada uma única vez, antes da primeira repetição.
- **Condição:** avaliada antes de cada repetição.
- **Atualização:** executada ao final de cada repetição.
- As três partes são separadas por ponto e vírgula.

Comando for: Exemplo

- **Problema:** imprimir todos os números inteiros de 1 até 100.

```
for (int i = 1; i <= 100; i++) {
    printf("%d\n", i);
}
```

- Ordem de execução:
  1. `i = 1;`
  2. Testa `i <= 100`
  3. Executa o bloco
  4. `i++;`
  5. Testa novamente

Comando for: Exemplo

- **Problema:** ler n e imprimir as n primeiras potências de 2, sem utilizar operador de exponenciação.

```c
#include <stdio.h>

int main() {
    int n, potencia = 1;
    printf("Digite um numero inteiro positivo: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        potencia *= 2;
        printf("%d\n", potencia);
    }

    return 0;
}
```

Definindo o Intervalo

- Os limites e o passo do laço ficam explícitos no cabeçalho:

```
for (int i = 0; i < 4; i++) {...}      // 0, 1, 2, 3
for (int i = 1; i <= 4; i++) {...}     // 1, 2, 3, 4
for (int i = 2; i < 10; i += 2) {...} // 2, 4, 6, 8
for (int i = 1; i < 15; i += 3) {...} // 1, 4, 7, 10, 13
for (int i = 5; i > 0; i--) {...}      // 5, 4, 3, 2, 1
```

Comando for × while

- Todo for pode ser reescrito como um while:

```
for (int i = 1; i <= n; i++) {
    printf("%d\n", i);
}
```

```
int i = 1;
while (i <= n) {
    printf("%d\n", i);
    i++;
}
```

- O for é preferido quando o número de repetições é conhecido.

Escopo do Contador

- Uma variável declarada na inicialização existe apenas dentro do laço:

```c
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
printf("%d\n", i); // error: 'i' undeclared
```

- Se o valor final do contador for necessário depois do laço, a declaração deve ficar fora dele:

```c
int i;
for (i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
printf("%d\n", i); // 6
```

Variações do Comando for

- As três partes do cabeçalho são **opcionais**, mas os dois pontos e vírgulas permanecem:

```
int i = 1;
for (; i <= 5; i++) { ... } // Sem inicialização

for (int i = 1; i <= 5; ) { i++; ... } // Sem atualização

for (;;) { ... } // Laço infinito, equivale a while (1)
```

- O **operador vírgula** permite mais de uma expressão na inicialização e na atualização:

```
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("%d %d\n", i, j);
}
```

Comandos `break` e `continue`

Comando break

- O **comando `break`** encerra imediatamente a estrutura de repetição, transferindo a execução para o comando seguinte ao laço.
- É útil quando o objetivo do laço já foi alcançado e continuar as repetições seria desnecessário.

```c
for (int i = 1; i < 10; i++) {
    if (i == 5) {
        break;
    }
    printf("%d\n", i); // 1, 2, 3, 4
}
```

Comando break: Exemplo

- **Problema:** contar quantos números inteiros positivos são fornecidos ao programa, até a leitura de um inteiro não positivo.

```c
#include <stdio.h>

int main() {
    int x, n = 0;

    while (1) {
        printf("Digite um numero inteiro positivo: ");
        scanf("%d", &x);

        if (x <= 0)
            break;
        n++;
    }

    printf("Quantidade de numeros positivos: %d\n", n);
    return 0;
}
```

Comando continue

- O **comando** `continue` encerra apenas a repetição atual e passa para a próxima.
- As instruções restantes do bloco são ignoradas, mas o laço continua.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    printf("%d\n", i); // 1, 2, 4, 5
}
```

Comando continue: Exemplo

- **Problema:** somar os números de 1 a 100, ignorando os múltiplos de 3.

```c
#include <stdio.h>

int main() {
    int soma = 0;

    for (int i = 1; i <= 100; i++) {
        if (i % 3 == 0)
            continue;
        soma += i;
    }

    printf("Soma: %d\n", soma); // 3367
    return 0;
}
```

Funções e Protótipos

Funções

- Uma **função** agrupa um conjunto de comandos que são executados quando ela é chamada.
- Em C, toda função possui um **tipo de retorno** e uma lista de **parâmetros tipados**, ambos declarados explicitamente.
- **Sintaxe:**

```
tipo_retorno nome_funcao(tipo_1 param_1, tipo_2 param_2,
    ...) {
    // Corpo da função
    return valor;
}
```

- Os **nomes** das funções seguem as mesmas regras dos nomes de variáveis.

Funções: Exemplo

```c
#include <stdio.h>

int soma(int x, int y) {
    return x + y;
}

int main() {
    int resultado = soma(4, 7);
    printf("%d\n", resultado); // 11

    printf("%d\n", soma(2, 3)); // 5
    return 0;
}
```

Tipo `void`

- O **tipo `void`** indica ausência de valor e aparece em dois papéis distintos.
- Como tipo de retorno, indica que a função não devolve valor algum:

```
void imprime_media(double media) {
    printf("Media: %.2lf\n", media);
}
```

- Na lista de parâmetros, indica que a função não recebe argumentos:

```
void imprime_cabecalho(void) {
    printf("Estruturas de Dados I\n");
}
```

- Uma função `void` não pode ter o seu resultado atribuído a uma variável.

Comando `return`

- O **comando `return`** devolve um valor e interrompe a execução da função.
- O valor devolvido deve ser compatível com o tipo declarado no cabeçalho.
- Uma função pode conter vários `return`, mas apenas um deles é executado:

```
int classifica(int x) {
    if (x > 0) {
        return 1;
    } else if (x < 0) {
        return -1;
    }
    return 0;
}
```

Comando return

- Em funções `void`, o `return` pode ser usado sem valor, apenas para encerrar a execução:

```c
void imprime_divisao(int a, int b) {
    if (b == 0) {
        printf("Divisao por zero\n");
        return;
    }
    printf("%d\n", a / b);
}
```

- Uma função retorna **um único valor**:

```c
int soma_e_subtracao(int x, int y) {
    return x + y, x - y; // Retorna apenas x - y
}
```

Ordem de Definição

- Uma função precisa ser conhecida antes de ser chamada:

```c
#include <stdio.h>

int main() {
    printf("%d\n", soma(4, 7));
    return 0;
}

int soma(int x, int y) {
    return x + y;
}
// error: implicit declaration of function 'soma'
```

- É preciso organizar o código-fonte pela ordem das chamadas.

Protótipos

- Um **protótipo** declara o cabeçalho da função, sem o corpo, e é encerrado por ponto e vírgula.
- Ele informa ao compilador o nome, o tipo de retorno e os tipos dos parâmetros, permitindo que a definição venha depois.

```c
#include <stdio.h>

int soma(int x, int y); // Protótipo

int main() {
    printf("%d\n", soma(4, 7));
    return 0;
}

int soma(int x, int y) { // Definição
    return x + y;
}
```

Protótipos

- Com protótipos, o código-fonte passa a ter uma estrutura padronizada:

```c
#include <stdio.h>                    // 1. Bibliotecas

#define MAX 100                       // 2. Constantes

int soma(int, int);                   // 3. Protótipos
double media(int, int);

int main() { ... }                    // 4. Função principal

int soma(int x, int y) { ... }        // 5. Definições
double media(int x, int y) { ... }
```

Funções e Protótipos: Exemplo

- **Problema:** ler um número inteiro $n > 1$ e informar se ele é primo.

```
#include <stdio.h>

int testa_primo(int n);

int main() {
    int n;
    printf("Digite um numero inteiro maior que 1: ");
    scanf("%d", &n);

    if (testa_primo(n))
        printf("%d e primo\n", n);
    else
        printf("%d nao e primo\n", n);

    return 0;
}
...
```

Funções e Protótipos: Exemplo

```
...
int testa_primo(int n) {
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Escopo de Variáveis

Variáveis Locais

- O **escopo** de uma variável é a região do programa em que ela pode ser acessada.
- Uma variável declarada dentro de uma função é **local**.
- Os parâmetros seguem a mesma regra das variáveis locais.

```c
void imprime_dobro(int x) {
    int dobro = 2 * x;
    printf("%d\n", dobro);
}

int main() {
    imprime_dobro(5);         // 10
    printf("%d\n", dobro);    // error: 'dobro' undeclared
    return 0;
}
```

Variáveis Globais

- Uma variável declarada fora de todas as funções é **global** e pode ser acessada por qualquer uma delas.

```c
#include <stdio.h>

int contador = 0; // Variável global

void incrementa() {
    contador++;
}

int main() {
    incrementa();
    incrementa();
    printf("%d\n", contador); // 2
    return 0;
}
```

- O uso de globais dificulta a manutenção: qualquer função pode alterá-las. Toda informação de que a função precisa deve chegar como parâmetro.

Escopo de Variáveis

- Quando uma variável local tem o mesmo nome de uma global, a local **"esconde"** a global dentro daquele escopo.

```c
#include <stdio.h>

int a = 1; // Global

void imprime() {
    int a = 5; // Local
    a++;
    printf("%d\n", a); // 6
}

int main() {
    imprime();
    printf("%d\n", a); // 1
    return 0;
}
```

Passagem de Parâmetros por Valor

Passagem por Valor

- Em C, os argumentos são passados **por valor**: a função recebe uma **cópia** do valor informado na chamada.
- Alterar o parâmetro dentro da função não afeta a variável usada na chamada.

```c
#include <stdio.h>

void incrementa(int x) {
    x = x + 1;
    printf("%d\n", x); // 2
}

int main() {
    int a = 1;
    incrementa(a);
    printf("%d\n", a); // 1
    return 0;
}
```

Passagem por Valor

Consequência

- Uma função só devolve resultados por meio do `return`.
- Como o `return` produz um único valor, **ainda** não é possível escrever uma função que altere duas variáveis do programa que a chamou.
- Retornos múltiplos passam a ser possíveis com **registros (`struct`) e ponteiros**.

Passagem por Valor: Exemplo

- **Problema:** trocar os valores de duas variáveis.

```c
void troca(int x, int y) {
    int aux = x;
    x = y;
    y = aux;
}

int main() {
    int a = 10, b = 20;
    troca(a, b);
    printf("%d %d\n", a, b); // 10 20
    return 0;
}
```

- O algoritmo está correto, mas atua sobre as **cópias**, que são destruídas ao final da função.

Exceção: Vetores

- Um **vetor** armazena vários valores de um mesmo tipo em posições consecutivas de memória, acessadas por um índice que começa em zero.

```c
int numeros[3] = {7, 8, 9};

printf("%d\n", numeros[0]); // 7
numeros[2] = 100;
```

- O tamanho é fixado na declaração e não é armazenado junto com o vetor.

Exceção: Vetores

- Vetores não são copiados na chamada: a função recebe uma indicação de onde o vetor está na memória e trabalha sobre o **original**.

```c
void zera_primeiro(int v[], int n) {
    v[0] = 0;
}

int main() {
    int numeros[3] = {7, 8, 9};

    zera_primeiro(numeros, 3);
    printf("%d\n", numeros[0]); // 0

    return 0;
}
```

Estruturas de Dados I

Estruturas Condicionais em C

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

26 de agosto de 2026

Objetivos da Aula

- Delimitar **blocos de comandos** na linguagem C.
- Implementar **estruturas condicionais** com `if`, `if-else` e cadeias `else if`.
- Reconhecer os **erros** mais frequentes na implementação de condicionais.
- Utilizar o **operador ternário** e o **comando `switch`**.

Blocos e Fluxo de Controle

Fluxo de Controle

- Por padrão, as instruções de um programa são executadas **em sequência**.
- As **estruturas condicionais** permitem desviar esse fluxo, selecionando quais instruções serão executadas.
- Em C, a condição é uma **expressão inteira**: o valor 0 corresponde a falso, e qualquer valor diferente de zero corresponde a verdadeiro.
- Qualquer expressão que produza um número pode ser usada como condição:

```c
int n = 5;

if (n) { ... }      // Verdadeiro
if (n % 2) { ... }  // Verdadeiro
if (n - 5) { ... }  // Falso
```

Blocos de Comandos

- Um **bloco** é um conjunto de comandos agrupados, delimitado por chaves.
- Sem as chaves, o `if` controla apenas a instrução **imediatamente seguinte**.
- Em C, a indentação **não possui** efeito sintático.

```c
if (media >= 6) {
    printf("Aprovado\n");
    printf("Continue assim!\n");
}
```

Escopo em Blocos

- Uma variável declarada dentro de um bloco existe **apenas** naquele bloco.
- Ao final do bloco, o espaço reservado para ela é liberado, e o nome deixa de ser reconhecido.

```c
if (x > 0) {
    int dobro = 2 * x;
    printf("%d\n", dobro);
}

printf("%d\n", dobro); // error: 'dobro' undeclared
```

- Variáveis que precisam ser usadas depois da estrutura condicional devem ser declaradas **fora** dela.

Comandos `if`, `else` e `else if`

Comando `if`

- O **comando `if`** executa um bloco somente quando a condição é verdadeira.

```
if (condicao) {
    // Bloco executado apenas se a condição for verdadeira
}
```

- Os **parênteses** ao redor da condição são obrigatórios.

Comando if: Exemplo

```c
#include <stdio.h>

int main() {
    double n1, n2, n3, n4, media;

    printf("Digite as quatro notas: ");
    scanf("%lf %lf %lf %lf", &n1, &n2, &n3, &n4);

    media = (n1 + n2 + n3 + n4) / 4;
    printf("Media final: %.2lf\n", media);

    if (media >= 6) {
        printf("Aprovado\n");
    }

    return 0;
}
```

Erros Comuns

- Sem chaves, apenas a primeira instrução pertence ao `if`:

```c
if (media >= 6)
    printf("Aprovado\n");
    printf("Continue assim!\n"); // Sempre executado
```

- Um ponto e vírgula após a condição encerra o comando, e o bloco passa a ser independente:

```c
if (media >= 6); // Comando vazio
{
    printf("Aprovado\n"); // Sempre executado
}
```

- Nos dois casos o programa compila sem erros, e a falha aparece somente no resultado.

Erros Comuns

- Atribuição no lugar da comparação:

```c
int aprovado = 0;

if (aprovado = 1) {
    printf("Aprovado\n"); // Sempre executado
}
```

- A atribuição é uma expressão e devolve o valor atribuído.
- Como 1 é diferente de zero, a condição nunca é falsa.
- Além disso, a variável `aprovado` é modificada pelo próprio teste.

Comando `if-else`

- O **bloco do `else`** é executado quando a condição é falsa.

```
if (condicao) {
    // Bloco executado apenas se a condição for verdadeira
} else {
    // Bloco executado apenas se a condição for falsa
}
```

Comando if-else: Exemplo

```c
#include <stdio.h>

int main() {
    double a, b;

    printf("Digite dois numeros: ");
    scanf("%lf %lf", &a, &b);

    if (a > b) {
        printf("O maior valor e %.2lf\n", a);
    } else {
        printf("O maior valor e %.2lf\n", b);
    }

    return 0;
}
```

Comando `if-else`

- A qual `if` pertence o `else`?

```
if (x > 0)
    if (y > 0)
        printf("Ambos positivos\n");
else
    printf("x nao e positivo\n");
```

Comando `if-else`

- A qual `if` pertence o `else`?

```
if (x > 0)
    if (y > 0)
        printf("Ambos positivos\n");
else
    printf("x nao e positivo\n");
```

- Sem chaves, o `else` se associa ao `if` **mais próximo**.
- Delimitar todos os blocos com **chaves** torna a associação explícita e elimina a ambiguidade na leitura do código.

Cadeias `else if`

- Para testar várias alternativas, **encadeia-se** um novo `if` dentro do `else`.

```
if (condicao_1) {
    // Bloco 1
} else if (condicao_2) {
    // Bloco 2
} else if (condicao_3) {
    // Bloco 3
} else {
    // Bloco executado se nenhuma condição for satisfeita
}
```

- As condições são avaliadas de cima para baixo, e a cadeia é encerrada na primeira que for verdadeira.
- O bloco do `else` final é **opcional**.

Cadeias `else if`: Exemplo

- **Problema:** aplicar um aumento salarial escalonado, de 15% para salários de até R$ 1.000,00, de 10% até R$ 2.000,00 e de 5% acima desse valor.
- Tentativa com comandos `if` independentes:

```
if (salario <= 1000) {
    salario = salario * 1.15;
}
if (salario > 1000 && salario <= 2000) {
    salario = salario * 1.10;
}
if (salario > 2000) {
    salario = salario * 1.05;
}
```

- Para um salário de R$ 950,00, o primeiro aumento resulta em R$ 1.092,50, valor que satisfaz a segunda condição e recebe um novo aumento.

Cadeias `else if`: Exemplo

- Com a cadeia, apenas um dos blocos é executado:

```
if (salario <= 1000) {
    salario = salario * 1.15;
} else if (salario <= 2000) {
    salario = salario * 1.10;
} else {
    salario = salario * 1.05;
}
```

Cadeias `else if`: Exemplo

- **Problema:** classificar o eleitor pela idade em *não eleitor* (menos de 16 anos), *eleitor facultativo* (de 16 a 17 anos e acima de 65 anos) e *eleitor obrigatório* (de 18 a 65 anos).

```
int idade;

printf("Digite a sua idade: ");
scanf("%d", &idade);

if (idade < 16) {
    printf("Nao eleitor\n");
} else if (idade < 18) {
    printf("Eleitor facultativo\n");
} else if (idade <= 65) {
    printf("Eleitor obrigatorio\n");
} else {
    printf("Eleitor facultativo\n");
}
```

Qual a Saída destes Trechos?

- **Trecho 1:**

```c
int a = 5;

if (a > 10)
    printf("A\n");
else if (a > 3)
    printf("B\n");
else if (a > 0)
    printf("C\n");
else
    printf("D\n");
```

Qual a Saída destes Trechos?

- **Trecho 1:**

```c
int a = 5;

if (a > 10)
    printf("A\n");
else if (a > 3)
    printf("B\n");
else if (a > 0)
    printf("C\n");
else
    printf("D\n");
```

- **Resposta:**

```c
// Saída: B
```

Qual a Saída destes Trechos?

- **Trecho 2:**

```c
int nota = 9;

if (nota >= 0)
    printf("Regular\n");
else if (nota >= 5)
    printf("Bom\n");
else if (nota >= 8)
    printf("Otimo\n");
```

Qual a Saída destes Trechos?

- **Trecho 2:**

```c
int nota = 9;

if (nota >= 0)
    printf("Regular\n");
else if (nota >= 5)
    printf("Bom\n");
else if (nota >= 8)
    printf("Otimo\n");
```

- **Resposta:**

```c
// Saída: Regular
```

Qual a Saída destes Trechos?

- **Trecho 3:**

```c
int x = 8;

if (x % 2 == 0)
    printf("Par ");
if (x % 4 == 0)
    printf("Multiplo de 4\n");
```

Qual a Saída destes Trechos?

- **Trecho 3:**

```c
int x = 8;

if (x % 2 == 0)
    printf("Par ");
if (x % 4 == 0)
    printf("Multiplo de 4\n");
```

- **Resposta:**

```c
// Saída: Par Multiplo de 4
```

Condicionais Aninhados

Condicionais Aninhados

- Um **condicional aninhado** é um `if` inserido dentro do bloco de outro `if` ou de um `else`.
- Permite verificar condições que só fazem sentido depois que uma condição mais ampla foi satisfeita.

```
if (condicao_externa) {
    if (condicao_interna) {
        // Bloco executado se ambas forem verdadeiras
    } else {
        // Externa verdadeira e interna falsa
    }
} else {
    // Bloco executado se a externa for falsa
}
```

Condicionais Aninhados: Exemplo

- **Problema:** informar se um número inteiro é positivo, negativo ou zero e, quando positivo, se é par ou ímpar.

```
int numero;
printf("Digite um numero inteiro: ");
scanf("%d", &numero);

if (numero > 0) {
    if (numero % 2 == 0) {
        printf("Positivo e par\n");
    } else {
        printf("Positivo e impar\n");
    }
} else if (numero < 0) {
    printf("Negativo\n");
} else {
    printf("Zero\n");
}
```

Aninhamento e Operadores Lógicos

- Quando o aninhamento serve apenas para exigir duas condições simultâneas, o **operador** `&&` produz o mesmo resultado:

```
if (idade >= 18) {
    if (possui_habilitacao) {
        printf("Pode dirigir\n");
    }
}
```

```
if (idade >= 18 && possui_habilitacao) {
    printf("Pode dirigir\n");
}
```

- O aninhamento continua necessário quando cada nível possui um `else` próprio.

Aninhamento e Legibilidade

- Cada nível de aninhamento acrescenta um recuo e uma condição a ser acompanhada pelo leitor.

```
if (usuario_existe) {
    if (senha_correta) {
        if (conta_ativa) {
            printf("Acesso concedido\n");
        } else {
            printf("Conta inativa\n");
        }
    } else {
        printf("Senha incorreta\n");
    }
} else {
    printf("Usuario nao encontrado\n");
}
```

Aninhamento e Legibilidade

- Aninhamentos com muitos níveis **dificultam** a leitura e a manutenção do código.
- **Soluções:**
  - Tratar primeiro os casos de erro e encerrar o fluxo;
  - Combinar condições independentes com operadores lógicos.

Operador Ternário e Comando `switch`

Operador Ternário

- O **operador ternário** escolhe entre dois valores a partir de uma condição.
- **Sintaxe:**

```
condicao ? valor_se_verdadeiro : valor_se_falso
```

- Adequado para escolhas simples entre dois valores, especialmente em **atribuições** e **saídas**.
- Esse operador **produz um valor** e pode ser usado dentro de expressões.

```
int a = 9, b = 2;

int maior = (a > b) ? a : b;       // 9
int absoluto = (a < 0) ? -a : a;   // 9
```

Operador Ternário: Exemplos

- Como devolve um valor, pode aparecer diretamente como argumento de `printf()`:

```c
double media = 7.5;

printf("%s\n", (media >= 6) ? "Aprovado" : "Reprovado");
// Aprovado
printf("%d\n", (media >= 6) ? 1 : 0); // 1
```

- O uso encadeado é possível, mas **reduz a legibilidade**:

```c
int n = -4;

printf("%d\n", (n > 0) ? 1 : (n < 0) ? -1 : 0); // -1
```

Comando switch

- O **comando switch** compara o valor de uma única expressão com uma lista de valores constantes.
- Substitui uma cadeia else if em que a **mesma variável** é testada repetidamente por igualdade.
- É frequente na escolha de opções de menu, na conversão de códigos e no tratamento de caracteres.

```
if (opcao == 1) {
    ...
} else if (opcao == 2) {
    ...
} else if (opcao == 3) {
    ...
} else {
    ...
}
```

Comando switch

- **Sintaxe:**

```
switch (expressao) {
    case valor_1:
        // Comandos executados se expressao == valor_1
        break;
    case valor_2:
        // Comandos executados se expressao == valor_2
        break;
    default:
        // Comandos executados se nenhum caso corresponder
}
```

Comando `switch`

- Cada **`case`** indica um valor a ser comparado, e os dois-pontos encerram o rótulo.
- O **comando `break`** encerra o `switch` e transfere a execução para depois das chaves.
- O **`default`** é opcional e corresponde ao `else` final de uma cadeia.
- Os blocos de cada caso não exigem chaves.

Comando switch: Exemplo

```
double a, b;
char op;

printf("Digite dois numeros: ");
scanf("%lf %lf", &a, &b);
printf("Operacao (+, -, *, /): ");
scanf(" %c", &op);

switch (op) {
    case '+':
        printf("%.2lf\n", a + b);
        break;
    case '-':
        printf("%.2lf\n", a - b);
        break;
...
```

Comando switch: Exemplo

```
...
    case '*':
        printf("%.2lf\n", a * b);
        break;
    case '/':
        printf("%.2lf\n", a / b);
        break;
    default:
        printf("Operacao invalida\n");
}
```

Ausência do break

- Encontrado o caso correspondente, a execução continua pelos casos seguintes até encontrar um `break` ou o final do `switch`.

```c
int opcao = 2;

switch (opcao) {
    case 1:
        printf("Um\n");
    case 2:
        printf("Dois\n");
    case 3:
        printf("Tres\n");
    default:
        printf("Opcao invalida\n");
}
```

- O esquecimento do `break` não gera erro de compilação.

Agrupamento de Casos

- A continuação da execução também pode ser usada de forma intencional:

```
int nota;
char conceito;
printf("Digite a nota (0 a 10): ");
scanf("%d", &nota);
switch (nota) {
    case 10:
    case 9:
        conceito = 'A';
        break;
    case 8:
    case 7:
        conceito = 'B';
        break;
    default:
        conceito = 'C';
}
printf("Conceito: %c\n", conceito);
```

Restrições do `switch`

- A **expressão** testada deve ser de um tipo inteiro, o que inclui `int` e `char`, mas exclui `float`, `double` e cadeias de caracteres.
- Os **rótulos dos casos** devem ser constantes, e não variáveis ou expressões.
- Não é possível expressar faixas de valores nem comparações com < e >.
- Não podem existir dois casos com o mesmo valor.

Restrições do switch

```
switch (media) {
    case 6.0:
        ...
}
// error: switch quantity not an integer

switch (idade) {
    case limite:
        ...
}
// error: case label does not reduce to an integer
    constant
```

Qual a Saída destes Trechos?

- **Trecho 1:**

```c
int x = 3;
switch (x) {
    case 1:
        printf("A ");
    case 3:
        printf("B ");
        break;
    case 4:
        printf("C ");
    default:
        printf("D ");
}
printf("\n");
```

Qual a Saída destes Trechos?

- **Trecho 1:**

```c
int x = 3;
switch (x) {
    case 1:
        printf("A ");
    case 3:
        printf("B ");
        break;
    case 4:
        printf("C ");
    default:
        printf("D ");
}
printf("\n");
```

- **Resposta:**

```c
// Saída: B
```

Qual a Saída destes Trechos?

- **Trecho 2:**

```c
char letra = 'e';
switch (letra) {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        printf("Vogal\n");
        break;
    default:
        printf("Consoante\n");
}
```

Qual a Saída destes Trechos?

- **Trecho 2:**

```c
char letra = 'e';
switch (letra) {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        printf("Vogal\n");
        break;
    default:
        printf("Consoante\n");
}
```

- **Resposta:**

```c
// Saída: Vogal
```

Qual a Saída destes Trechos?

- **Trecho 3:**

```c
int n = 7;
switch (n % 3) {
    case 0:
        printf("Multiplo de 3\n");
        break;
    case 1:
        printf("Resto 1\n");
    case 2:
        printf("Resto 2\n");
}
```

Qual a Saída destes Trechos?

- **Trecho 3:**

```c
int n = 7;
switch (n % 3) {
    case 0:
        printf("Multiplo de 3\n");
        break;
    case 1:
        printf("Resto 1\n");
    case 2:
        printf("Resto 2\n");
}
```

- **Resposta:**

```c
/* Saída:
Resto 1
Resto 2 */
```

Estruturas de Dados I

Vetores, Matrizes e Cadeias de Caracteres

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

2 de setembro de 2026

Objetivos da Aula

- Declarar, inicializar e percorrer **vetores**.
- Estender a ideia para duas dimensões com **matrizes**.
- Representar textos como **cadeias de caracteres**.
- Utilizar **funções** da biblioteca padrão para manipular cadeias de caracteres.

Vetores

Motivação

- Até agora, cada variável armazena um **único valor** por vez.
- Guardar as notas de cinco alunos, por exemplo, exigiria cinco variáveis independentes:

```
float nota1, nota2, nota3, nota4, nota5;
```

- O problema cresce com a quantidade de dados: com 100 alunos, a solução deixa de ser viável.

Vetor

- Um **vetor ou *array* unidimensional** agrupa vários valores do mesmo tipo em posições consecutivas de memória.
- Todo o conjunto recebe um **único nome**, e cada valor é alcançado por um **índice**.
- O acesso por índice é **direto**: o custo de alcançar qualquer posição é o mesmo.

Declaração

- A **declaração** informa o tipo dos elementos, o nome do vetor e a quantidade de posições:

```
tipo nome[quantidade];
```

- **Exemplos:**

```
float notas[5];
int idades[100];
char letras[26];
```

- Todos os elementos de um vetor têm obrigatoriamente o **mesmo tipo**.

Declaração

- O tamanho do vetor é definido na declaração e **não muda** durante a execução.
- O tamanho deve ser uma constante conhecida antes da execução:

```
#define MAX 50

int main() {
    int v[MAX];
    int w[10];
    ...
    return 0;
}
```

- Na prática, declara-se um vetor com **capacidade suficiente** e utiliza-se apenas parte dele.
- **Não existem** operações para inserir ou remover elementos.

Inicialização

- Os valores iniciais podem ser fornecidos entre **chaves**, na própria declaração:

```
int numeros[5] = {10, 20, 30, 40, 50};
```

- Se a lista tiver menos valores que o tamanho declarado, as posições restantes recebem **zero**:

```
int v[5] = {1, 2};    // {1, 2, 0, 0, 0}
int z[100] = {0};     // Todas as posições recebem zero
```

- Quando a lista é fornecida, o tamanho pode ser **omitido** e o compilador o deduz:

```
int primos[] = {2, 3, 5, 7, 11}; // 5 posições
```

Acesso aos Elementos

- Cada posição é **acessada** pelo nome do vetor seguido do índice entre colchetes.
- Os índices vão de 0 até `tamanho - 1`.

```c
int v[5] = {10, 20, 30, 40, 50};

printf("%d\n", v[0]); // 10
printf("%d\n", v[4]); // 50

v[2] = 99;
v[3] = v[0] + v[1];
```

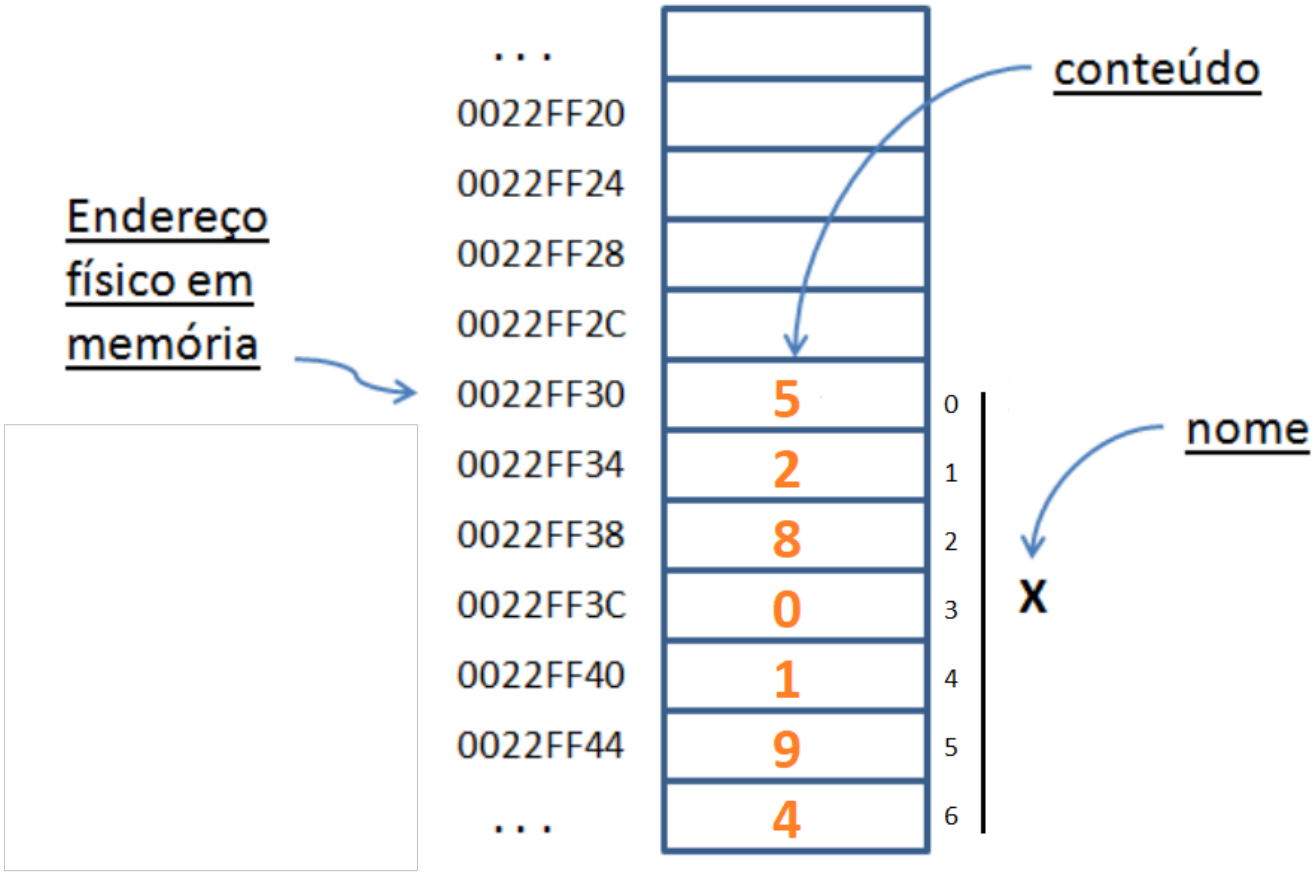
Organização na Memória

- As posições ocupam **endereços consecutivos**, na ordem dos índices.
- O **espaço total** é o tamanho de um elemento multiplicado pela quantidade de posições.
- **Exemplo:**

```
int X[7] = {5, 2, 8, 0, 1, 9, 4};
```

Espaço total $= 4$ *bytes* $\times 7$ posições $= 28$ *bytes*.

Organização na Memória



X[0]	X[1]	X[2]	X[3]	X[4]	X[5]	X[6]
5	2	8	0	1	9	4

Percorrendo um Vetor

- O índice de um vetor pode ser uma variável, o que permite **percorrer** todas as posições com um laço:

```
int v[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++) {
    printf("v[%d] = %d\n", i, v[i]);
}
```

- O laço percorre **índices**, e não elementos.
- A **leitura** também é feita posição a posição, com o operador & aplicado ao elemento:

```
float notas[5];
for (int i = 0; i < 5; i++) {
    printf("Nota %d: ", i + 1);
    scanf("%f", &notas[i]);
}
```

Ausência de Verificação de Limites

- C não verifica se o índice utilizado está dentro do vetor.
- Um índice inválido não gera erro: o programa lê ou escreve em uma região de memória que não pertence ao vetor.

```c
int v[5] = {10, 20, 30, 40, 50};

printf("%d\n", v[7]);   // Valor imprevisível
v[10] = 1;              // Escreve fora do vetor
```

- O programa pode produzir valores errados, sobrescrever outras variáveis ou encerrar abruptamente.

Cópia e Comparação

- O operador de atribuição **não copia** o conteúdo de um vetor para outro:

```
int a[3] = {1, 2, 3};
int b[3];

b = a; // error: assignment to expression with array type
```

- Os operadores relacionais também **não comparam** os elementos.
- A cópia e a comparação são feitas **posição a posição**:

```
for (int i = 0; i < 3; i++) {
    b[i] = a[i];
}
```

Vetores como Parâmetro de Função

- Um vetor passado a uma função não é copiado: a função recebe a indicação de onde ele está e trabalha sobre o **original**.
- Como o tamanho não acompanha o vetor, ele é passado em um parâmetro adicional.

```
void dobra(int v[], int n) {
    for (int i = 0; i < n; i++)
        v[i] = v[i] * 2;
}

int main() {
    int v[3] = {1, 2, 3};
    dobra(v, 3);
    printf("%d\n", v[0]);
    return 0;
}
```

Vetores: Exemplo

- **Problema:** Leia as notas de uma turma, calcule a média e identifique a maior delas.

```c
#include <stdio.h>
#define TAM 5

int main() {
    float notas[TAM];
    float soma = 0.0;

    for (int i = 0; i < TAM; i++) {
        printf("Nota %d: ", i + 1);
        scanf("%f", &notas[i]);
        soma = soma + notas[i];
    }
    ...
```

Vetores: Exemplo

```
    ...
    float maior = notas[0];

    for (int i = 1; i < TAM; i++) {
        if (notas[i] > maior) {
            maior = notas[i];
        }
    }

    printf("Media: %.2f\n", soma / TAM);
    printf("Maior nota: %.2f\n", maior);
    return 0;
}
```

Matrizes

Matriz

- Nem todo conjunto de dados é naturalmente organizado em uma única sequência.
- As notas de uma turma em várias avaliações, por exemplo, formam uma tabela, com alunos nas linhas e avaliações nas colunas.
- Representar essa tabela com vetores exigiria um vetor por avaliação.
- Uma **matriz** é um vetor cujos elementos também são vetores.
- O conjunto passa a ter **duas dimensões**, e cada posição é localizada por dois índices.

Declaração

- A **declaração** informa o tipo dos elementos e a quantidade de linhas e de colunas, cada uma em seu par de colchetes:

```
tipo nome[linhas][colunas];
```

- **Exemplos:**

```
float notas[40][4];
int tabuleiro[3][3];
char mapa[10][20];
```

- As duas dimensões são **fixas**.
- Todos os elementos são do **mesmo tipo**.
- Todas as linhas têm exatamente o **mesmo comprimento**.

Representação

- Os **índices** de linha variam de `0` até `linhas - 1`, e os de coluna, de `0` até `colunas - 1`.
- Índices de uma matriz declarada como `int m[3][4]`:

$$\mathtt{m} = \begin{bmatrix} \mathtt{m[0][0]} & \mathtt{m[0][1]} & \mathtt{m[0][2]} & \mathtt{m[0][3]} \\ \mathtt{m[1][0]} & \mathtt{m[1][1]} & \mathtt{m[1][2]} & \mathtt{m[1][3]} \\ \mathtt{m[2][0]} & \mathtt{m[2][1]} & \mathtt{m[2][2]} & \mathtt{m[2][3]} \end{bmatrix}$$

Inicialização

- Os valores iniciais são fornecidos em **chaves aninhadas**, uma para cada linha:

```
int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};
```

- As chaves internas podem ser **omitidas**, e os valores são distribuídos linha a linha:

```
int m[2][3] = {1, 2, 3, 4, 5, 6}; // Mesmo resultado
```

- Posições não informadas recebem **zero**:

```
int m[2][3] = {0}; /* {{0, 0, 0},
                      {0, 0, 0}} */
```

Acesso aos Elementos

- Cada posição é **acessada** com os dois índices em pares de colchetes separados:

```
int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};

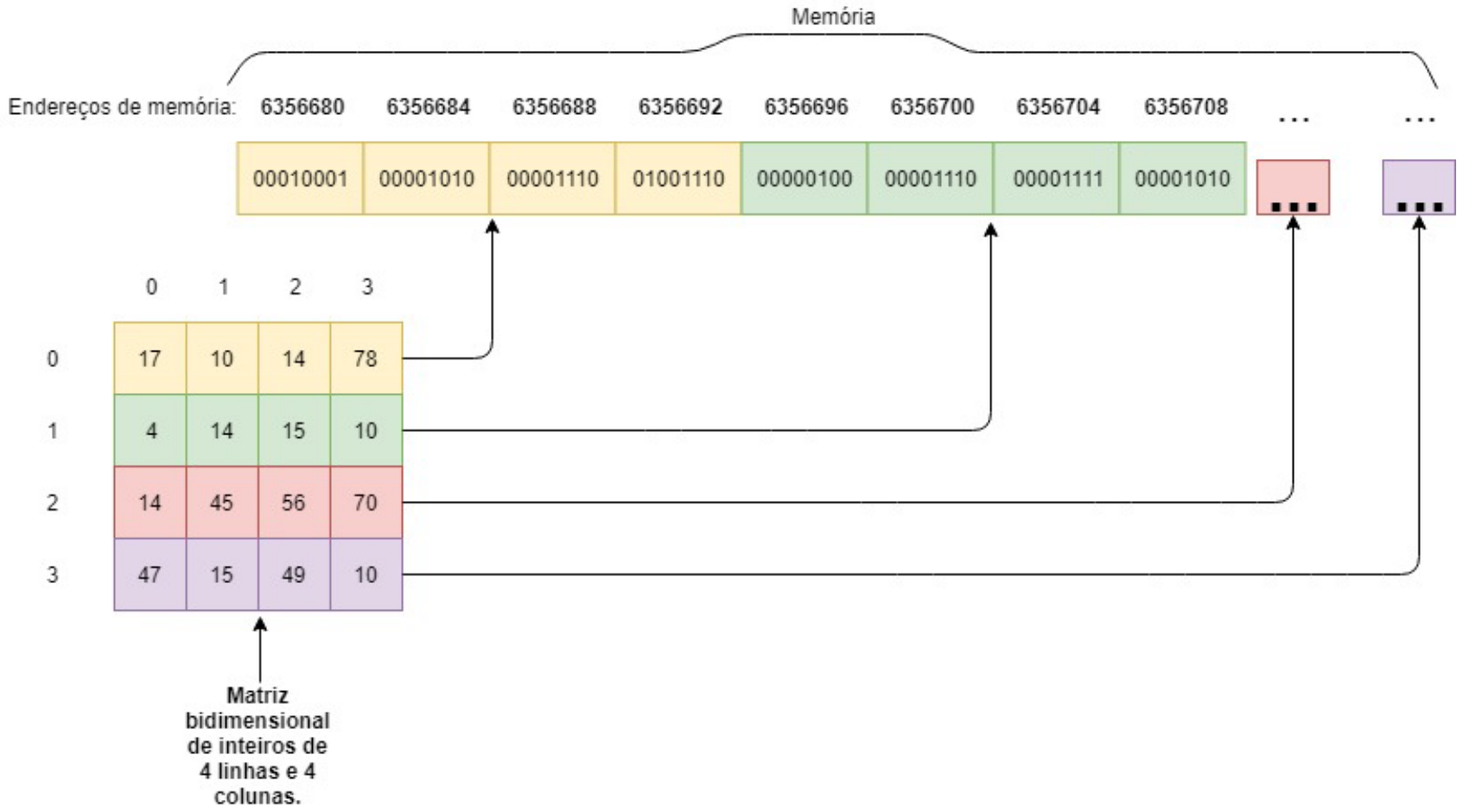
printf("%d\n", m[0][0]);   // 1
printf("%d\n", m[1][2]);   // 6

m[0][1] = 99;
m[1][0] = m[0][0] + m[0][2];
```

Organização na Memória

- A memória é **linear**, de modo que a matriz é armazenada como uma sequência única, com as linhas dispostas uma após a outra.
- O elemento `m[i][j]` está a `i * colunas + j` posições do início.

Organização na Memória



Percorrendo uma Matriz

- O **percurso** de uma matriz exige dois laços aninhados:

```
int m[2][3] = {{1, 2, 3},
               {4, 5, 6}};

for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        printf("%d ", m[i][j]);
    }
    printf("\n");
}
```

Leitura de uma Matriz

- A **leitura** segue a mesma estrutura de laços aninhados, e o operador & é aplicado ao elemento:

```
float notas[3][2];

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        printf("Aluno %d, prova %d: ", i + 1, j + 1);
        scanf("%f", &notas[i][j]);
    }
}
```

Uso Parcial da Matriz

- Quando as dimensões dependem da entrada do usuário, declara-se a matriz com um **limite máximo** e utiliza-se apenas **parte** dela:

```c
#define MAX 100

int main() {
    int m[MAX][MAX];
    int linhas, colunas;
    scanf("%d %d", &linhas, &colunas);
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    return 0;
}
```

Cópia e Comparação

- Assim como nos vetores, **não** é possível copiar ou comparar matrizes diretamente:

```
int a[2][2] = {{1, 2}, {3, 4}};
int b[2][2];

b = a; // error: assignment to expression with array type
```

- A cópia e a comparação são feitas **elemento a elemento**, com laços aninhados:

```
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        b[i][j] = a[i][j];
    }
}
```

Matriz como Parâmetro de Função

- A matriz também não é copiada na chamada: a função trabalha sobre a **original**.
- O número de colunas é **obrigatório** na declaração do parâmetro, pois sem ele não é possível localizar `m[i][j]`.
- O número de linhas continua sendo passado como **parâmetro adicional**.

```
void imprime(int m[][3], int linhas) {
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
}
```

Matrizes: Exemplo

- **Problema:** Leia duas matrizes de mesmas dimensões e calcule a soma das matrizes.

```
#include <stdio.h>
#define MAX 10

int main() {
    int a[MAX][MAX], b[MAX][MAX], c[MAX][MAX];
    int linhas, colunas;

    printf("Linhas e colunas: ");
    scanf("%d %d", &linhas, &colunas);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            scanf("%d", &a[i][j]);
    ...
```

Matrizes: Exemplo

```
    ...
    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            scanf("%d", &b[i][j]);

    for (int i = 0; i < linhas; i++)
        for (int j = 0; j < colunas; j++)
            c[i][j] = a[i][j] + b[i][j];

    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }

    return 0;
}
```

Arrays Multidimensionais

- A linguagem **não impõe** um limite de duas dimensões: basta acrescentar pares de colchetes.
- Cada dimensão adicional exige um índice a mais no acesso e um laço a mais no percurso.
- O armazenamento continua linear.

```
int dados[2][2][2] = {{{1, 2}, {3, 4}},
                      {{5, 6}, {7, 8}}};
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        for (int k = 0; k < 2; k++) {
            printf("%d ", dados[i][j][k]);
        }
        printf("\n");
    }
    printf("\n");
}
```

Cadeias de Caracteres

Textos em C

- C **não possui** um tipo próprio para texto.
- Uma **cadeia de caracteres ou *string*** é um vetor de `char`, em que o fim do texto é marcado pelo caractere nulo `'\0'`.
- Como cadeias de caracteres são vetores, todas as regras para vetores são **válidas**.

Caractere '\0'

- As funções que manipulam texto **não recebem** o tamanho da cadeia: elas percorrem as posições até encontrar '\0'.
- O vetor precisa ter uma posição a mais do que o número de caracteres do texto.

0	1	2	3	4	5	6	7	8
p	r	o	g	r	a	m	a	\0

- As posições após o terminador existem, mas são ignoradas.

0	1	2	3	4	5	6	7	8
c	a	s	a	\0	?	?	?	?

Declaração e Inicialização

- A forma mais direta de inicialização usa um literal entre **aspas duplas**, e o terminador é acrescentado automaticamente:

```
char nome[10] = "Ana";
char curso[] = "Ciencia de Dados"; // Tamanho deduzido
```

- A inicialização também pode ser feita **caractere a caractere**, e nesse caso o terminador é responsabilidade do programador:

```
char nome[4] = {'A', 'n', 'a', '\0'};
```

Atribuição após a Declaração

- O literal entre aspas duplas só pode ser usado no momento da declaração:

```
char nome[10] = "Ana";    // Válido

char cidade[10];
cidade = "Londrina";      // error: assignment to
    expression with array type
```

- Depois da declaração, o conteúdo pode ser **alterado** posição a posição ou por uma função da biblioteca padrão.

```
char nome[10] = "Ana";
nome[0] = 'E'; // "Ena"
```

Imprimindo uma Cadeia de Caracteres

- O **especificador %s** imprime os caracteres a partir do início até encontrar o terminador:

```
char nome[10] = "Ana";
printf("Nome: %s\n", nome);
```

- A **função `puts()`** imprime a cadeia e acrescenta uma quebra de linha:

```
puts(nome);
```

Lendo uma Cadeia de Caracteres

- O **especificador `%s`** também lê cadeias, e o operador `&` não é utilizado:

```c
char nome[20];

printf("Nome: ");
scanf("%s", nome);
```

- A leitura para no primeiro espaço em branco.
- Uma entrada maior que o tamanho declarado escreve fora da região de memória reservada.
- O terminador é inserido automaticamente ao final.

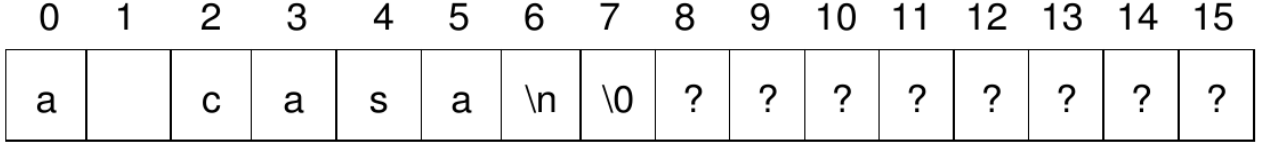
Lendo uma Cadeia de Caracteres

- A **função `fgets()`** lê uma linha inteira:

```
char frase[100];

printf("Frase: ");
fgets(frase, 100, stdin);
```

- Os três argumentos são o vetor de destino, o tamanho disponível e a origem da leitura, que é a entrada padrão.
- Espaços são preservados, e a leitura nunca ultrapassa o tamanho informado.
- A tecla `Enter` também é armazenada: a cadeia lida termina em `'\n'` seguido de `'\0'`.



Percorrendo uma Cadeia

- No percurso de uma cadeia, a condição do laço testa o terminador.
- Cada posição é acessada por um índice, como em qualquer vetor.

```
char frase[100] = "Estruturas";

for (int i = 0; frase[i] != '\0'; i++) {
    printf("%c\n", frase[i]);
}
```

Biblioteca `string.h`

- As **operações mais comuns** sobre cadeias estão na biblioteca `string.h` e exigem a inclusão do cabeçalho:

```
#include <string.h>
```

Tabela 1: Funções mais utilizadas da biblioteca `string.h`

Função	**Finalidade**
`strlen()`	Comprimento da cadeia
`strcpy()`	Cópia de uma cadeia para outra
`strcat()`	Concatenação de duas cadeias
`strcmp()`	Comparação de duas cadeias

Função `strlen()`

- Retorna a quantidade de caracteres antes do terminador:

```c
char nome[20] = "Ana";

printf("%zu\n", strlen(nome)); // 3
printf("%zu\n", sizeof(nome)); // 20
```

- O resultado é calculado percorrendo a cadeia a cada chamada, de modo que usar `strlen()` na condição de um laço repete esse percurso a cada repetição.

Função strcpy()

- Copia o conteúdo da segunda cadeia para a primeira, incluindo o terminador:

```
char origem[20] = "Estruturas";
char destino[20];

strcpy(destino, origem);
strcpy(destino, "Dados");    // Conteúdo anterior
    descartado

printf("%s\n", destino);     // "Dados"
```

- O vetor de destino precisa ter espaço para o texto e para o terminador.

Função strcat()

- Acrescenta a segunda cadeia ao final da primeira, a partir do terminador:

```c
char frase[50] = "Estruturas";

strcat(frase, " de ");
strcat(frase, "Dados");

printf("%s\n", frase); // "Estruturas de Dados"
```

- O vetor de destino precisa comportar a concatenação dos dois textos mais o terminador.

Função `strcmp()`

- Os operadores relacionais não comparam o conteúdo de duas cadeias, e a comparação é feita por `strcmp()`:

```
char a[20] = "casa";
char b[20] = "casa";

if (strcmp(a, b) == 0) {
    printf("São iguais\n");
}
```

- **Retorno:**
  - Zero quando as cadeias são iguais;
  - Negativo quando a primeira vem antes em ordem alfabética;
  - Positivo quando a segunda vem antes em ordem alfabética.

Vetores de Cadeias

- Um conjunto de textos é representado por uma **matriz de `char`**: cada linha guarda uma cadeia.

```
char nomes[3][20] = {"Ana", "Bruno", "Carla"};

for (int i = 0; i < 3; i++) {
    printf("%s\n", nomes[i]);
}
```

- Todas as linhas ocupam o mesmo espaço.

Cadeias como Parâmetro de Função

- Como qualquer vetor, a cadeia não é copiada na chamada: a função trabalha sobre o **original**.

```
int conta_letra(char s[], char c) {
    int total = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == c) {
            total++;
        }
    }
    return total;
}
```

Cadeias de Caracteres: Exemplo

- **Problema:** Leia uma frase e conte quantas vogais ela contém.

```c
#include <stdio.h>
#include <string.h>
#define MAX 100

int main() {
    char frase[MAX];
    int vogais = 0;

    printf("Frase: ");
    fgets(frase, MAX, stdin);
    ...
```

Cadeias de Caracteres: Exemplo

```
    ...
    int n = strlen(frase);
    if (n > 0 && frase[n - 1] == '\n') {
        frase[n - 1] = '\0';
    }

    for (int i = 0; frase[i] != '\0'; i++) {
        char c = frase[i];
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u') {
            vogais++;
        }
    }

    printf("A frase tem %d vogais\n", vogais);
    return 0;
}
```

Estruturas de Dados I

Registros (`struct`)

Professor: Gustavo T. Naozuka
gtnaozuka@uel.br



Universidade
Estadual de Londrina

9 de setembro de 2026

Objetivos da Aula

- Compreender a necessidade de agrupar dados de **tipos diferentes**.
- Definir e utilizar **registros** por meio da construção `struct`.
- Criar nomes de tipos com **`typedef`**.
- Construir vetores de registros e registros aninhados.
- Passar registros para funções e retorná-los.

Introdução

Motivação

- O vetor resolve o problema de armazenar **muitos valores do mesmo tipo** sob um único nome:

```
float notas[50];
```

- Existe, porém, o problema oposto: **poucos valores de tipos diferentes** que descrevem uma mesma entidade.
- Um aluno, por exemplo, possui nome, matrícula e notas. São informações de naturezas distintas, mas que pertencem ao mesmo indivíduo.

Motivação

- Até agora, o cadastro de um aluno seria escrito assim:

```
char nome[50];
int matricula;
float nota1, nota2;
```

- A solução funciona para um aluno, mas apresenta **limitações** importantes:
  - Não há indicação de que essas quatro variáveis pertencem ao mesmo aluno;
  - Cada função que trate de um aluno precisa receber os quatro parâmetros;
  - Para 50 alunos, seriam necessários quatro vetores independentes.

Motivação

- O mesmo padrão se repete em outras situações:

```
// Fração
int numerador;
int denominador;

// Data
int dia, mes, ano;

// Ponto no plano
float x, y;
```

Registros

Registro

- Um **registro** é um conjunto de variáveis agrupadas sob um único nome. Em C, ele é definido com a **construção `struct`**:

```
struct Aluno {
    char nome[50];
    int matricula;
    float nota1;
    float nota2;
};
```

- Cada variável interna é chamada de **campo** ou **membro** do registro.
- Os campos podem ser de **qualquer tipo** válido em C.
- Os campos de um registro ocupam posições **consecutivas** de memória, na ordem da definição.

Registro

- O exemplo anterior descreve como é um aluno: apenas foi criado um novo tipo, chamado `struct Aluno`.
- Para obter uma **variável** desse tipo, é preciso declará-la:

```
struct Aluno a;          // Variável
struct Aluno turma[50]; // Vetor de registros
```

- A palavra `struct` acompanha o nome escolhido e **não pode** ser omitida.

Acesso aos Campos

- Os campos de um registro são acessados pelo **operador ponto**:

```c
struct Aluno a;

strcpy(a.nome, "Ana Souza");
a.matricula = 20260145;
a.nota1 = 8.5;
a.nota2 = 7.0;

printf("Aluno: %s\n", a.nome);
printf("Matricula: %d\n", a.matricula);
printf("Media: %.2f\n", (a.nota1 + a.nota2) / 2);
```

- Cada campo se comporta como uma variável do seu próprio tipo.
- A impressão do registro é feita **campo a campo**.

Inicialização

- Um registro pode ser inicializado no momento da declaração, com os valores entre **chaves**:

```
struct Aluno a = {"Ana Souza", 20260145, 8.5, 7.0};
```

- Campos omitidos recebem **zero**:

```
struct Aluno b = {"Bruno Lima", 20260146};
// nota1 e nota2 recebem 0.0
```

- Os valores são associados aos campos **na ordem** em que foram definidos na `struct`.
- Se um campo novo for inserido no meio da definição, todas as inicializações passam a atribuir valores errados.

Inicialização

- A partir do padrão C99, é possível **nomear** o campo que recebe cada valor:

```
struct Aluno a = {
    .nome = "Ana Souza",
    .matricula = 20260145,
    .nota1 = 8.5,
    .nota2 = 7.0
};
```

- A ordem passa a ser **livre** e o código indica explicitamente o destino de cada valor.
- Campos não mencionados continuam recebendo **zero**.

Atribuição

- A **atribuição** entre registros do mesmo tipo é válida e copia todos os campos:

```
struct Aluno a = {"Ana Souza", 20260145, 8.5, 7.0};
struct Aluno b;

b = a; // Copia os 4 campos
```

- As variáveis `a` e `b` permanecem independentes.

Local de Definição

- A `struct` é definida **fora** de qualquer função, logo após as diretivas do pré-processador.
- Uma `struct` definida dentro da função `main()` tem escopo **local**.

```c
#include <stdio.h>

struct Ponto {
    float x;
    float y;
};

void imprimir(struct Ponto p);

int main() { ... }
```

Leitura e Escrita de um Registro

```c
#include <stdio.h>

struct Aluno {
    char nome[50];
    int matricula;
    float nota1;
    float nota2;
};

int main() {
    struct Aluno a;
    float media;

    printf("Nome: ");
    fgets(a.nome, 50, stdin);
    ...
```

Leitura e Escrita de um Registro

```
    ...
    printf("Matricula: ");
    scanf("%d", &a.matricula);

    printf("Primeira nota: ");
    scanf("%f", &a.nota1);

    printf("Segunda nota: ");
    scanf("%f", &a.nota2);

    media = (a.nota1 + a.nota2) / 2;

    printf("\nAluno: %s", a.nome);
    printf("Matricula: %d\n", a.matricula);
    printf("Media: %.2f\n", media);
    return 0;
}
```

Comando `typedef`

Comando typedef

- O **comando `typedef`** cria um nome alternativo para um tipo já existente:

```
typedef tipo_existente novo_nome;
```

- Pode ser aplicado a qualquer tipo da linguagem:

```
typedef unsigned int Positivo;
typedef char Texto[50];

Positivo idade = 21;
Texto nome = "Ana Souza";
```

- Pode ser utilizado com **registros**.

Comando typedef

- Definido o registro, o typedef atribui um nome curto ao tipo struct Aluno:

```
struct Aluno {
    char nome[50];
    int matricula;
    float nota1;
    float nota2;
};

typedef struct Aluno Aluno;
```

- A palavra struct deixa de ser necessária na declaração de variáveis:

```
Aluno a;
Aluno turma[50];
```

Forma Condensada

- A definição do registro e a criação do nome do tipo podem ser escritas em um **único comando**:

```
typedef struct {
    char nome[50];
    int matricula;
    float nota1;
    float nota2;
} Aluno;
```

- O nome que aparece antes do ; final é o nome do **tipo**.

Convenção de Nomes

- Nomes de tipos criados com `typedef` serão escritos com **inicial maiúscula**; variáveis, com inicial minúscula:

```
typedef struct {
    int dia, mes, ano;
} Data;

Data nascimento;
Data feriados[12];
```

- A convenção permite distinguir o que é tipo e o que é variável.

Necessidade do Nome do Registro

- A forma condensada não serve quando um campo do registro precisa se referir ao **próprio tipo** que está sendo definido.
- Nesse caso, o nome do registro é **indispensável**:

```
typedef struct No {
    int valor;
    struct No *proximo;
} No;
```

- Essa estrutura será retomada nas próximas aulas, quando o significado do campo `proximo` for estudado.

Vetores de Registros e Registros Aninhados

Vetores de Registros

- Um registro pode ser o tipo dos elementos de um **vetor**. Cada posição armazena um registro completo:

```
Aluno turma[50];
```

- O **acesso** combina o índice do vetor com o operador ponto:

```
strcpy(turma[0].nome, "Ana Souza");
turma[0].matricula = 20260145;
turma[0].nota1 = 8.5;
turma[0].nota2 = 7.0;

printf("Aluno: %s\n", turma[0].nome);
printf("Matricula: %d\n", turma[0].matricula);
printf("Notas: %.2f e %.2f\n", turma[0].nota1,
turma[0].nota2);
```

Preenchendo um Vetor de Registros

```c
#include <stdio.h>

#define N 3

typedef struct {
    char nome[50];
    int matricula;
    float nota1;
    float nota2;
} Aluno;

int main() {
    Aluno turma[N];
    ...
```

Preenchendo um Vetor de Registros

```
    ...
    for (int i = 0; i < N; i++) {
        printf("Aluno %d\n", i + 1);

        printf("Nome: ");
        fgets(turma[i].nome, 50, stdin);

        printf("Matricula: ");
        scanf("%d", &turma[i].matricula);

        printf("Notas: ");
        scanf("%f %f", &turma[i].nota1, &turma[i].nota2);

        getchar(); // Descarta o Enter deixado pelo scanf
    }
    return 0;
}
```

Percorrendo um Vetor de Registros

```c
#include <stdio.h>

#define N 3

typedef struct {
    char nome[50];
    int matricula;
    float nota1, nota2;
} Aluno;

int main() {
    Aluno turma[N] = {{"Ana Souza",   20260145, 8.5, 7.0},
                      {"Bruno Lima",  20260146, 6.0, 9.5},
                      {"Carla Dias",  20260147, 9.0, 8.0}
    };
    float media, soma = 0, maior = turma[0].nota1;
    ...
```

Percorrendo um Vetor de Registros

```
    ...
    for (int i = 0; i < N; i++) {
        media = (turma[i].nota1 + turma[i].nota2) / 2;
        soma = soma + media;

        if (turma[i].nota1 > maior) maior = turma[i].nota1;
        if (turma[i].nota2 > maior) maior = turma[i].nota2;

        printf("%s %d media %.2f\n", turma[i].nome,
               turma[i].matricula, media);
    }

    printf("\nMedia da turma: %.2f\n", soma / N);
    printf("Maior nota: %.2f\n", maior);

    return 0;
}
```

Registros Aninhados

- Um campo de registro pode ser também um registro:

```
typedef struct {
    int dia;
    int mes;
    int ano;
} Data;

typedef struct {
    char nome[50];
    int matricula;
    Data nascimento;
} Aluno;
```

- Em registros aninhados, a **ordem** importa.

Acesso Encadeado

- Para alcançar um campo interno, o operador ponto é aplicado **sucessivamente**, da estrutura mais externa para a mais interna:

```
Aluno a;

a.nascimento.dia = 12;
a.nascimento.mes = 3;
a.nascimento.ano = 2005;

printf("%02d/%02d/%d\n", a.nascimento.dia,
a.nascimento.mes, a.nascimento.ano);
```

- Na **inicialização**, o registro interno aparece entre seu próprio par de chaves:

```
Aluno b = {"Bruno Lima", 20260146, {25, 11, 2004}};
```

Registros e Funções

Registros como Parâmetro

- Um registro pode ser passado como **parâmetro** para uma função:

```
float media(Aluno a) {
    return (a.nota1 + a.nota2) / 2;
}
```

- A chamada passa o **registro inteiro**, e não os campos separadamente:

```
printf("Media: %.2f\n", media(turma[0]));
```

Registros como Parâmetro: Exemplo

```c
#include <stdio.h>

#define N 2

typedef struct {
    char nome[50];
    int matricula;
    float nota1, nota2;
} Aluno;

float media(Aluno a) {
    return (a.nota1 + a.nota2) / 2;
}
```

Registros como Parâmetro: Exemplo

```c
void imprimir(Aluno a) {
    printf("%s (%d)\n", a.nome, a.matricula);
    printf(" Media: %.2f\n", media(a));
}

int main() {
    Aluno turma[N] = {
        {"Ana Souza", 20260145, 8.5, 7.0},
        {"Bruno Lima", 20260146, 6.0, 9.5}
    };

    for (int i = 0; i < N; i++) {
        imprimir(turma[i]);
    }

    return 0;
}
```

Retorno de Registros

- Uma função também pode **retornar** um registro completo:

```
Data montar_data(int d, int m, int a) {
    Data nova;
    nova.dia = d;
    nova.mes = m;
    nova.ano = a;
    return nova;
}
```

- O valor retornado pode ser **atribuído** diretamente a uma variável ou a um campo:

```
Aluno aluno;
aluno.nascimento = montar_data(12, 3, 2005);
```

- Essa é uma vantagem sobre os vetores, que não podem ser retornados por uma função.

Passagem de Parâmetros

- A passagem de parâmetros continua sendo **por valor**. A função trabalha sobre uma cópia do registro:

```
void aprovar(Aluno a) {
    a.nota1 = 10.0; // Altera apenas a cópia
    a.nota2 = 10.0;
}

int main() {
    Aluno x = {"Ana Souza", 20260145, 8.5, 7.0};

    aprovar(x);
    printf("%.2f %.2f\n", x.nota1, x.nota2); // 8.50 7.00

    return 0;
}
```

Custo da Cópia

- Copiar um registro é **mais caro** do que copiar uma variável simples.
- Existem, portanto, duas **limitações** abertas:
  - A função não consegue alterar o registro original;
  - Registros grandes são copiados integralmente a cada chamada.
- As duas limitações têm a mesma solução: **passagem de parâmetros por referência**.

Erros Comuns

Erros Comuns

- A definição de um registro é um comando e termina com ;

```
typedef struct {
    int dia, mes, ano;
} Data

int main() { // error: two or more data types in
    declaration specifiers
    ...
}
```

Erros Comuns

- Não existe comparação direta entre dois registros:

```
if (a == b) // error: invalid operands to binary == (have
    'Aluno' and 'Aluno')
```

- A comparação precisa ser feita campo a campo:

```
if (a.matricula == b.matricula) {
    printf("Mesmo aluno\n");
}

if (d1.dia == d2.dia && d1.mes == d2.mes &&
    d1.ano == d2.ano) {
    printf("Mesma data\n");
}
```